CAPSTONE PROJECT: FINAL REPORT

IndieGuard: Indie Game Launch Analytics

Predicting negative-review risk and identifying winning feature combinations on Steam

Course	23CSE452 Business Analytics (Units 1, 2 and 3)
Programme / section	[programme and section]
Team number	Group 1
Guide	Rohini S
Academic year	2026-2027
Submission date	[submission date]
Repository and board	github.com/Shan713/IndieGuard; github.com/users/Shan713/projects/1
Live dashboard	https://indieguard.streamlit.app

Team member	Register number
Thotakura Srihitha	CB.SC.U4CSE23450
Gayas Sheik	CB.SC.U4CSE23118
Shantharam P	CB.SC.U4CSE23146
Pradeep Sasidharan	CB.SC.U4CSE23232
Valikala Tejaswini	CB.SC.U4CSE23752

Declaration and acknowledgement

We declare that this report and the project it describes are our own work as a team, with AI assistance disclosed in Section 8.3. The data were collected by our own scrapers from public Steam sources; no pre-built dataset was used. Prior studies, algorithms and software are cited in the reference list. Our use of generative-AI tools, what each tool supported, the code and content it affected, and how we verified and corrected it are disclosed in full in Section 8.3. Tools assisted the work; the project's research design, data collection runs, analytical decisions and final conclusions were made, run and checked by the team.

We thank our guide, Rohini S, for guidance and feedback during the capstone, and our course faculty for the structure and rubric that shaped the work.

Team member	**Signature**	**Date**
Thotakura Srihitha		
Gayas Sheik		
Shantharam P		
Pradeep Sasidharan		
Valikala Tejaswini		

Abstract

A small indie studio has little room for a bad launch. Steam lets a player refund a game bought in the last 14 days if they have played it for under two hours, and early impressions shape a game's rating. Refunds are not public, so we do not measure them; instead we study negative reviews written in that first two-hour window as a proxy for launch risk. IndieGuard asks what drives negative reviews, which feature combinations go with strong reception, and whether patches help. We collected our own data with throttled, resumable scrapers on 1,861 indie roguelike and roguelite games (2022 to 2025), 1.65 million reviews and 29,918 patch and sale events, cleaned them under 32 logged rules and split them by game. Exploratory analysis (Unit 1), five model families at review and game level with grid-search tuning (Unit 2), and two Unit 3 methods, text mining and association rule mining, were evaluated on held-out games. Reviews written inside the refund window are 21.2% negative against 8.8% later. A text model detects negative reviews with a PR-AUC of 0.84 and ROC-AUC of 0.97, but store metadata predicts a game's success tier only modestly (macro-F1 about 0.42 to 0.44 against 0.22 for guessing). Ninety-five tag combinations pass false-discovery control and about two-thirds of the testable ones hold on unseen games; patches show no clear effect against matched controls. A deployed Streamlit dashboard integrates the results and a launch-risk scorer. All findings are associations in observational data.

Executive summary

Small studios have to decide what to fix before launch, which feature combinations to back and when a patch is worth the effort, but public information gives them little to go on. We focus on negative reviews written in a game's first two hours, the period in which Steam still allows a refund. Refund numbers are not public and we do not measure them; an early negative review is used as a proxy, a sign that the first hour did not work for that player.

To study this we built our own dataset from Steam (1,861 indie roguelike games and 1.65 million reviews) and checked each result on games that were held back from the models. The main findings are:

- **The first two hours matter.** 21.2% of reviews written under two hours of play are negative, against 8.8% later; they make up 29% of all negative reviews. For games priced $20 or more the early figure is 31.8%.
- **Complaints change.** Early negative reviews lean to crashes, multiplayer bugs and controls; later ones to boss design, balance and repetition.
- **Store data alone say little.** A model that sees only store information finds a game's success tier about twice as well as guessing, which is useful for comparing launch plans but not a forecast.
- **Reading the review text works.** A text model flags negative reviews far better than store data (ROC-AUC 0.97 against 0.76 on the same reviews). It detects complaints; it does not forecast them.
- **Focused 2D, pixel, retro and procedural combinations** reach the Strong tier about two-thirds of the time (35% overall); no combination passed our thresholds for struggling launches.
- **Patches do not lift ratings by themselves.** Against matched games with no patch, the change is +0.27 points (interval -0.82 to +1.36).

From these results we suggest five actions for a studio to test, in priority order: (1) treat the first hour as a standalone product and blind-test it; (2) match the price to what the first hour delivers; (3) try a focused scope; (4) complete the store page; (5) patch for specific problems rather than to lift sentiment in general. In the illustrative scenario in Section 6.3, a first-hour fix that brings early negativity down to the level of cheap games would avoid about 14 negative reviews per 1,000; this is an estimate under stated assumptions, not a measured result. The live dashboard (https://indieguard.streamlit.app) lets a studio explore the market, the tag rules, the complaint themes, the patch results and score a planned game.

Table of contents

Declaration and acknowledgement ii

Abstract iii

Executive summary iv

Table of contents v

List of figures vii

List of tables vii

Abbreviations viii

1. Introduction, Business Objectives and Literature Review 1

    1.1 Context and scope 1

    1.2 Related analytics work and research gap 2

2. Dataset Collection, Validation and Preparation 2

    2.1 Provenance, coverage and challenges 2

    2.2 Data quality, privacy and ethics 3

    2.3 Transformations and usable dataset 4

3. Exploratory Data Analysis and Visualization (Unit 1) 4

4. Feature Engineering and Predictive Modelling (Unit 2) 5

5. Advanced Analytics (Unit 3): Two Selected Methods 7

    5.1 Method 1: Text mining 7

    5.2 Method 2: Association rule mining 9

6. Comparative Analysis and Business Impact 10

    6.1 Comparison of the two Unit 3 methods 10

    6.2 Comparison with existing studies 11

    6.3 KPI-based business impact and scenarios 11

7. Interactive Application, Figure Index and Reproducibility 12

    7.1 Mandatory dashboard 12

7.2 Figure index and reproduction map 13
7.3 Code and environment 13
8. End-to-End Validation, Traceability and Contributions 14
8.1 Completion and validity 14
8.2 Project tracking and individual work 14
8.3 AI use and originality declarations 15
9. Conclusion, Recommendations and Roadmap 15
References 16
Appendix A. Extended data dictionary and validation records 18
Appendix B. Tuning, additional figures and supplementary analysis 19
Appendix C. Implementation details and dashboard user guide 20
Appendix D. Tracking-board evidence, contribution records and AI declaration 21

List of figures

Figure 1. End-to-end workflow: how data collection, Units 1 to 3 and the dashboard connect. 2
Figure 2. Negative-review share by playtime at review and by price inside the 2-hour window (Python/Matplotlib; src/analysis, notebook 02_EDA_Visualizations.ipynb). 5
Figure 3. Cross-validation and test scores for all models at review level, game level and with review text (Python/Matplotlib; python -m src.models.compare). 6
Figure 4. Game-level XGBoost confusion matrices on the 299 test games (default and tuned class scales). 7
Figure 5. SHAP importance of the game-level XGBoost model (top features). 7
Figure 6. Confusion matrix of the text detector on the 120,487 held-out reviews (counts and row-normalised shares; Python/Matplotlib; python -m src.models.review_text_detail). 8
Figure 7. Complaint themes among negative reviews by playtime at review (NMF, eight topics; Python/Matplotlib; notebook 03). 9
Figure 8. The eight Strong rules with the highest lift: share of Strong games on training and test games (Python/Matplotlib; reports/results/association_rules_strong.csv). 10
Figure 9. Market explorer, default state (1,490 games). 12
Figure 10. Market explorer after selecting the tag Pixel Graphics (550 games; KPIs and charts update). 12
Figure 11. Tag rules page filtered to stable rules with lift at least 1.5 (37 of 95 rules). 13
Figure 12. Scorer explanation for a game predicted Struggling: reasons for and against, matched rules, and single changes that would help. 13
Figure 13. Patch impact by launch timing (change in positive-review share, points; Python/Matplotlib; notebooks/review2/patch_impact.ipynb). 20
Figure 14. Share of Struggling games by price band, release year, languages, DLC, achievements and review volume with 95% intervals (Python/Seaborn; notebooks/02_EDA_Visualizations.ipynb). 20

List of tables

Table 1. Business objectives, KPIs and success criteria. 1
Table 2. Representative sample of the final data: five games (top) and five reviews (bottom); full samples are in data/sample/. 3

Table 3. Mapping of key final fields to units and methods. 4
Table 4. Descriptive statistics of the main game- and review-level variables (cleaned data). 4
Table 5. Model comparison on the shared held-out test games (metadata only; guessing PR-AUC 0.106, macro-F1 0.22). Validation: 5-fold CV by game; test scored once. 6
Table 6. Text evaluation on 120,487 held-out English reviews (guessing PR-AUC 0.115). 8
Table 7. Association-rule configuration comparison (same 1,191 training and 299 test games for every row). 9
Table 8. Comparison of the two selected Unit 3 methods (their metrics are not interchangeable). ..10
Table 9. Comparison with existing studies (direct numerical comparison is not valid: different data, targets and protocols). 11
Table 10. Illustrative scenarios for a first-hour fix (hypothetical effect sizes and cost; sensitivity to the share of reviews written inside the window). 12
Table 11. Figure index and reproduction map (all figures are Python/Matplotlib or Seaborn and are embedded once). 13
Table 12. Objective-to-evidence traceability matrix. 14
Table 13. Individual contributions, dates and evidence (dates from the pull-request history). 14
Table 14. Prioritised recommendations and implementation roadmap (in priority order). 15
Table 15. Concise data dictionary of the cleaned tables. 18
Table 16. Key cleaning flags. 18
Table 17. Grid-search results: default (grid point 0) against tuned settings on the shared folds. 19
Table 18. Subgroup error check on the held-out test data (game scorer: macro-F1; text detector: PR-AUC; python -m src.models.subgroup_check). 19
Table 19. Project-board cards by member. 21
Table 20. Generative-AI disclosure (complete). 21

Abbreviations

Abbreviation	Meaning
AUC	Area under the curve (ROC-AUC: receiver operating characteristic; PR-AUC: precision-recall)
CV	Cross-validation
EDA	Exploratory data analysis
FDR	False discovery rate
FP-Growth	Frequent-pattern growth (itemset mining)
LDA	Latent Dirichlet allocation

Abbreviation	Meaning
NMF	Non-negative matrix factorization
PCA	Principal component analysis
SHAP	Shapley additive explanations
SVD	Singular value decomposition
TF-IDF	Term frequency, inverse document frequency

1. Introduction, Business Objectives and Literature Review

1.1 Context and scope

Steam lets players refund a game within 14 days of purchase if they have played it for less than two hours [4]. Refunds themselves are not public, but reviews are, and a negative review written in that window is an early sign that the first hour did not convince the player. Our stakeholders are indie studios and small publishers who must decide what to build, what to fix before launch and when a patch is worth it. We kept the scope narrow: indie roguelike and roguelite games released between 2022 and 2025 with at least 10 reviews (1,861 games). All findings are associations, and revenue and refund figures are out of scope because Steam does not publish them.

Overall objective: help a studio decide which feature combinations to back, what to fix in the first hours of play, and when to patch. We ask five questions: will a review be negative and how does playtime affect this (Q1); which success tier will a game reach (Q2); which tag, price and Early Access combinations go with strong reception (Q3); what do players complain about (Q4); and how do ratings change after a patch (Q5). Table 1 lists the KPIs and Figure 1 shows how the units fit together.

Table 1. Business objectives, KPIs and success criteria.

Business objective	KPI / unit	Baseline / target	Unit / method
O1. Quantify the refund-window effect (Q1)	Negative share inside vs outside the window (percentage points)	Baseline: 10.4% overall. Target: report the gap with a 95% interval	U1
O2. Detect negative reviews (Q1)	PR-AUC on held-out games	Baseline: guessing, 0.115. Target: clearly above baseline, CV spread reported	U2, U3 M1
O3. Predict the game tier (Q2)	Macro-F1 and recall per tier on held-out games	Baseline: 0.22. Target: clearly above baseline	U2
O4. Find tag combinations tied to Strong reception (Q3)	Rules passing FDR control ($q < 0.10$, lift at least 1.2) and holding on test games	Baseline: none. Target: rules with a held-out check	U3 M2
O5. Find complaint themes (Q4)	Topic diversity, coherence, stability and agreement with a manual check	Baseline: LDA topic model. Target: interpretable, stable themes	U3 M1
O6. Measure patch impact (Q5)	Change in positive-review share against matched no-patch controls (points)	Baseline: 0 points. Target: interval around the difference	U1, extra
O7. Deliver an interactive application	Public dashboard with filters that update results	Target: working link, 5 pages, live scenario	App

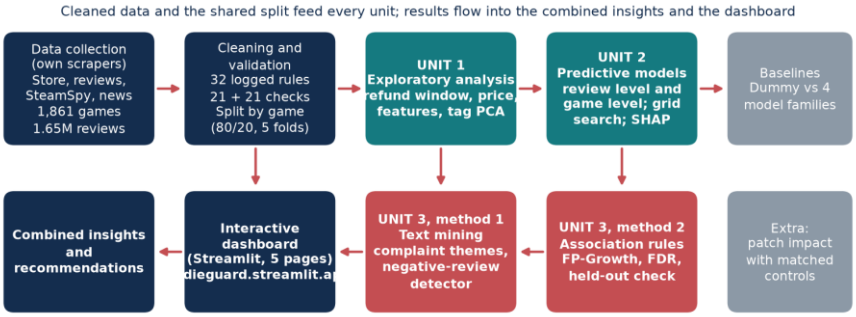


Figure 1. End-to-end workflow: how data collection, Units 1 to 3 and the dashboard connect.

1.2 Related analytics work and research gap

Lin et al. studied about 11 million reviews of 6,224 Steam games and found that negative reviews arrive early and that players criticise design more than bugs [1]; their work is descriptive. De Luisa et al. related price, size, languages, release date and genre to player numbers with Bayesian models [2], without review sentiment or text. Jorge and Pardo built SteamBR, 2.8 million Brazilian-Portuguese reviews, to predict review helpfulness [3]. Steam's review labels and SteamSpy's owner estimates [4, 5] describe games only after the fact. None of these links early-playtime negativity, launch features and patch timing with held-out testing. Text mining and association rules fit this gap: complaints live in free text, and launch decisions are combinations of tags, price and scope.

2. Dataset Collection, Validation and Preparation

2.1 Provenance, coverage and challenges

We collected all data ourselves on 1 and 2 October 2026 from four public Steam sources with scrapers we wrote: store search (Indie plus five roguelike tags, 2022 to 2025), store details and tags, SteamSpy owner estimates, and the news feed (patch and sale announcements); reviews were paged with Steam's cursor. No repository dataset or synthetic record is used. On usage terms, the Steam Web API Terms of Use [22] cap use at 100,000 calls a day and our scrapers, at one request every 1.25 to 1.6 seconds, stayed below it; the review endpoint is the one Steam documents publicly [4]; and SteamSpy asks for under one request per second [5], so we waited 1.1 seconds between calls. Our use is non-commercial and academic. Raw files stay on team machines, frozen with row counts and a hash, and are not redistributed; the public repository holds only the cleaned, pseudonymised tables.

The raw data hold 1,861 games (58 variables), 1,652,999 reviews in 31 languages and 29,918 news events; 1,648,387 reviews (34 variables) remain after cleaning. Data are observed at review, game

and announcement level. The review target is whether a review is negative (about 10% are); the game target is a tier for the 1,490 games with at least 20 reviews: Struggling (30% or more negative), Solid, and Strong (10% or less). Table 2 shows sample rows and Appendix A the data dictionary.

Table 2. Representative sample of the final data: five games (top) and five reviews (bottom); full samples are in data/sample/.

Game	Release year	Price	Steam reviews	Negative share	Tier
Cross The World	2025	$19.99	23	0%	Strong
Bloody Heaven 2	2024	$4.99	138	15%	Solid
Firework War	2025	$0.00	55	4%	Strong
Godrop	2024	$8.99	313	5%	Strong
From The Top	2025	$3.99	148	16%	Solid

Review text (shortened)	Language	Playtime at review	Target
I want Nemesis to fold me in half. 11/10 would...	english	6.8 h	positive
If you're reading comments right now to decide...	english	23.9 h	positive
great roguelike. its humerous and charming, bu...	english	3.1 h	positive
bad performance for a 2d game 5 fps in the upg...	english	0.9 h	negative
Hardness for the sake of hardness. Game has se...	english	11.6 h	negative

Steam started refusing requests (HTTP 429) after roughly 150 calls, so we made the scraper wait between requests, back off when refused and resume from where it stopped. SteamSpy was blocked by our network provider and had to be reached through another route. For the five largest games we kept only the newest 15,000 reviews; for every other game we collected almost all the reviews Steam reports.

2.2 Data quality, privacy and ethics

Cleaning followed 32 logged rules (counts in Appendix A). We dropped only unusable rows, mainly 4,612 empty reviews (0.28%), corrected 318 wrong launch dates and found no duplicate IDs. Suspicious records (very short or copy-pasted text, pre-launch reviews, extreme playtimes, free copies) were flagged rather than deleted. Playtime is missing for 12 reviews; an empty achievement count means none (22% of games); the Metacritic score exists for only 40 games, so we did not use it. Since only about 10% of reviews are negative, always guessing positive already scores about 89% accuracy.

Reviewer IDs are replaced by a salted hash and names, avatars and profile links are never stored, in line with the data-minimisation and pseudonymisation principles of India's Digital Personal Data

Protection Act, 2023 [6]; outputs are aggregated or at game level. Known biases: about 40% of reviews are in English, games with under 10 reviews are excluded, reviewers choose whether to review, store attributes are an October 2026 snapshot, and the five largest games are capped.

2.3 Transformations and usable dataset

Cleaning is kept separate from model-specific preprocessing. The main transformations were UTC timestamps, a winsorised copy of playtime, corrected launch dates, negation handling in text (for example *not_fun*), a PCA of the tag columns (Section 4) and TF-IDF with SVD for review text. Data are split by game, 80/20, with five cross-validation folds inside the training games; everything learned from data is fitted on training games only, and 21 automated checks test for leakage. Table 3 maps the main fields to each unit.

Table 3. Mapping of key final fields to units and methods.

Variable	Type / meaning	Origin / preparation	Used in
`voted_up`	bool: recommended or not	Steam review field; target = not voted_up	U1, U2, U3 M1
`author_playtime_at_review`	number (minutes)	Playtime when written; refund window = under 120 min	U1, U2, U3 M1
`review`, `language`	text, category	As written; English subset flagged `text_mining_ok`	U3 M1
`price_usd`, `is_free`, `n_dlc`	number, bool	Store details; price tier	U1, U2, U3 M2
`tags`, `genres`, `categories`	text lists	User tags (top 20); 68 PCA components; 137 rule items	U2, U3 M2
`steam_total_reviews/negative`	number	Steam totals; negative share and game tier (outcome, never a feature)	U1, U2 target
`events` (patch, sale)	event table	Steam news, classified; 29,918 rows	U1, extra

3. Exploratory Data Analysis and Visualization (Unit 1)

The exploratory work focused on when negative reviews appear (Q1) and how games differ at launch (Q3); figures were made with Matplotlib [20] and Seaborn [21]. Most variables are strongly skewed (Table 4): the median game has 50 Steam reviews but the largest more than 266,000, and the median review is five words long, written after about nine hours of play. We therefore use medians, shares and bands rather than means.

Table 4. Descriptive statistics of the main game- and review-level variables (cleaned data).

Variable	n	Mean	Median	Min	Max
Game price (USD)	1,861	7.91	5.99	0.00	199.99
Steam reviews per game	1,861	1,292	50	10	266,105
Negative share per game (%)	1,861	16.8	13.8	0.0	90.0
Supported languages	1,861	6.4	3	1	103
Achievements	1,861	28.1	20	0	330
Playtime at review (hours)	1,648,375	26.1	9.4	0.1	19,081.2

Variable	n	Mean	Median	Min	Max
Review length (words)	1,648,387	28.7	5	1	2,632

About 10.4% of reviews are negative, but reviews written in the first two hours are 21.2% negative, against roughly 8 to 10% later (Figure 2, left); they make up 29% of all negative reviews, and the pattern holds in 94% of games with enough reviews. Inside this window, negativity rises with price, from 11% under $5 to 23% at $5 to $10 and 32% at $20 or more (Figure 2, right). At game level, 14.5% of games are Struggling and 35% Strong.

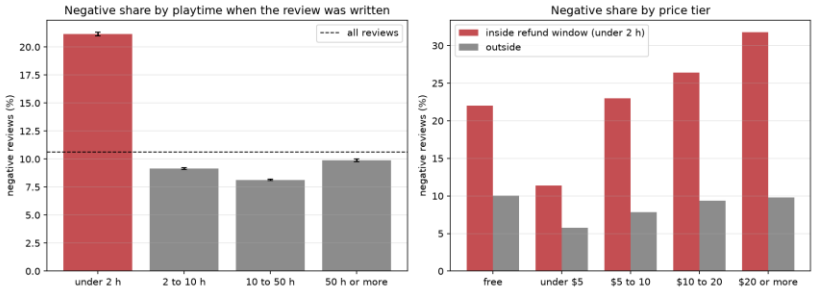


Figure 2. Negative-review share by playtime at review and by price inside the 2-hour window (Python/Matplotlib; src/analysis, notebook 02_EDA_Visualizations.ipynb).

Store attributes shift risk by a few points each and none decides it alone (Figure 14, Appendix B); for example, 12.5% of games listing Steam Achievements are Struggling against 19.6% of those without. Review counts, owner estimates and refund shares result from reception, so we kept them out of the models.

These observations shaped Unit 2. With only about 10% of reviews and 14.5% of games in the class of interest, accuracy rewards ignoring them, so we use PR-AUC and macro-F1. Reviews of one game share vocabulary, so a random split would let a model recognise games; we split by game. The playtime pattern suggested that the text itself carries the complaint, which led to the text model in Section 5.1.

4. Feature Engineering and Predictive Modelling (Unit 2)

At review level we predict whether a review is negative from playtime, refund-window and launch-week flags, days since launch, recent patches and sales, price and text length. At game level we predict the tier from 83 features known at launch: price, launch year, DLC, achievements, platforms, controller support and 68 tag components; outcome columns are never features. The 234 tag columns are sparse and strongly related (Pixel Graphics and Retro often appear together), so we compressed them with a PCA fitted on training games; its elbow at 68 components keeps 75% of the tag variance.

Each task has a Dummy baseline and four models: Logistic Regression, Random Forest, LightGBM and XGBoost [7, 8, 9], on the same split and folds. Each was tuned by grid search on the folds, keeping the default as a candidate (Appendix B, Table 17); thresholds and class scales come from out-of-fold predictions, and the test set was scored once. Leakage audit: outcome columns are excluded, no game is on both sides of the split, all reviews of a game sit in one fold (so folds and the tuning sample are grouped by game), and imputation and scaling are fitted inside each training fold; 21 automated checks pass. One exception: the text model's TF-IDF vocabulary and scaler (Section 5.1) were fitted on all training reviews before cross-validation, which can make its CV scores slightly optimistic but does not affect its test scores.

Table 5. Model comparison on the shared held-out test games (metadata only; guessing PR-AUC 0.106, macro-F1 0.22). Validation: 5-fold CV by game; test scored once.

Model	Review CV PR-AUC	Test PR-AUC	Test ROC-AUC	Game CV macro-F1	Test macro-F1 [95% CI]	Recall Struggling
Dummy (prior)	0.107 ± 0.024	0.106	0.500	0.225 ± 0.003	0.220 [0.20, 0.24]	0.00
Logistic Regression	0.208 ± 0.024	0.194	0.663	0.422 ± 0.033	0.423 [0.36, 0.48]	0.39
Random Forest	0.286 ± 0.038	0.280	0.726	0.434 ± 0.037	0.371 [0.31, 0.43]	0.10
LightGBM	0.287 ± 0.037	0.254	0.724	0.445 ± 0.027	0.372 [0.31, 0.43]	0.22
XGBoost	0.292 ± 0.037	0.275	0.729	0.444 ± 0.027	0.437 [0.38, 0.50]	0.41

At review level (Table 5, Figure 3) the tree models tie at a test PR-AUC of 0.25 to 0.28, about 2.5 times guessing; Logistic Regression is weaker (0.19). At game level every model beats guessing, but the intervals overlap with only 299 test games. Tuning helped XGBoost (0.437) and Logistic Regression (0.423) on test, while Random Forest and LightGBM fell to about 0.37 although their CV scores rose: with 1,191 training games, the search fitted noise in the folds, so we judge models on test results only. Struggling games are the hardest class (Figure 4). We selected XGBoost for its test score and Struggling recall, though it is not clearly better than Logistic Regression.

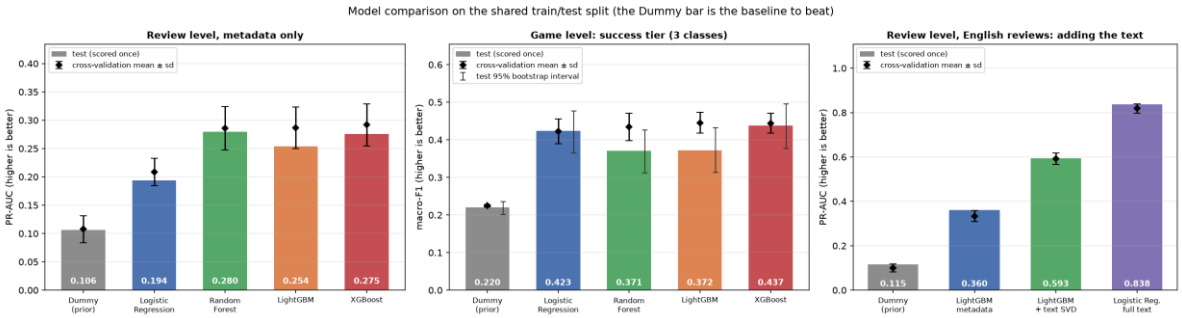


Figure 3. Cross-validation and test scores for all models at review level, game level and with review text (Python/Matplotlib; python -m src.models.compare).

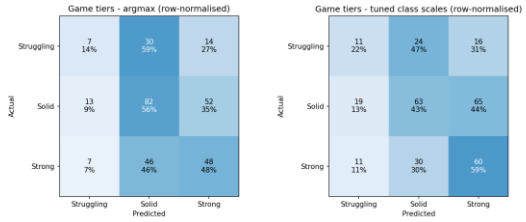


Figure 4. Game-level XGBoost confusion matrices on the 299 test games (default and tuned class scales).

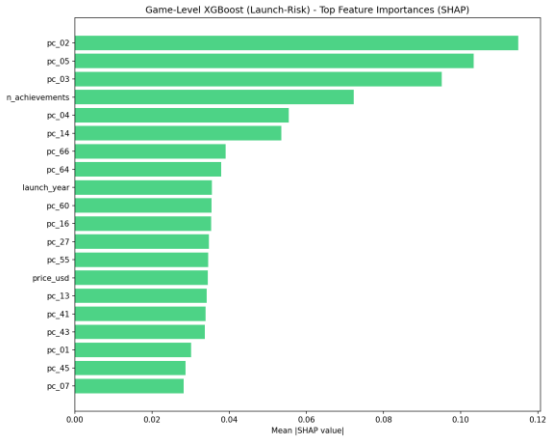


Figure 5. SHAP importance of the game-level XGBoost model (top features).

SHAP values [15] show that the review model leans on review length, Chinese-language reviews and playtime, and the game model on a few tag components and achievements (Figure 5). These are associations, and the review models use reviewer history collected at scrape time, so they explain reviews rather than predict a launch. Results are not driven by a few large games: the within-game ROC-AUC (0.75) is slightly above the pooled value.

To check for uneven errors, we sliced the test predictions by subgroup (Appendix B, Table 18). The deployed game scorer (macro-F1 0.43) is weaker for Early Access launches (0.33) and 2022 to 2023 games (0.36) than for 2024 to 2025 games (0.47); with 37 to 239 games per group the differences are uncertain, but Early Access plans deserve more caution. The text detector is stable across price bands (PR-AUC 0.83 to 0.86) and best inside the refund window (0.91), but weaker on reviews under 10 words (PR-AUC 0.73) and less precise on free games (0.60); it covers English only, so languages could not be compared.

5. Advanced Analytics (Unit 3): Two Selected Methods

The EDA left two questions for Unit 3. Negative reviews cluster in the first hours, but only the text says what players dislike, so we chose text mining. Store data predicted tiers only modestly, and a launch plan is a combination of tags, price and Early Access rather than one feature, so we chose association rules. Time-series analysis was not chosen because most games have too few reviews per week; patch impact is an extra analysis in Section 6.1.

5.1 Method 1: Text mining

Text mining asks what players complain about and whether the text can flag a negative review. We used 557,202 English reviews (436,715 training, 120,487 test). Negations were joined to the next

word, so *not fun* is one token. TF-IDF [10] weights words that are frequent in a review but rare overall; it suits short, informal text and linear models, and was learned from training reviews only.

On this representation we built a detector, a Logistic Regression on TF-IDF plus metadata tuned by grid search, with its threshold set on out-of-fold predictions; a model on 78 SVD components (the variance elbow) shows how much signal survives compression. We also fitted an NMF topic model [11] with eight topics on 40,000 negative training reviews and compared it with LDA [12]; eight was fixed so each topic could be read and labelled, not searched formally.

Table 6. Text evaluation on 120,487 held-out English reviews (guessing PR-AUC 0.115).

Approach	CV PR-AUC	Test PR-AUC	Test ROC-AUC	Precision / recall	F1
Dummy (prior)	0.101 ± 0.018	0.115	0.500	0.12 / 1.00	0.21
LightGBM, metadata only	0.334 ± 0.025	0.360	0.758	0.32 / 0.49	0.39
LightGBM, metadata + text components	0.592 ± 0.026	0.593	0.895	0.52 / 0.62	0.56
Logistic Regression, full text + metadata	0.819 ± 0.021	0.838	0.971	0.75 / 0.79	0.77

Compressed text lifts PR-AUC from 0.36 to 0.59 and full TF-IDF reaches 0.84 (ROC-AUC 0.97, Table 6). On the test reviews the detector catches 10,980 negative reviews and misses 2,893, with 3,607 false alarms (Figure 6); accuracy is 94.6% against 88.5% for always guessing positive. NMF scored lower than LDA on UMass coherence (-2.38 against -1.73), because UMass rewards co-occurring words and LDA repeated general gaming words across topics; NMF topics overlapped less (90% unique top words against 86%) and stayed similar on 80% resamples (mean Jaccard 0.75), so we used NMF. Early negative reviews lean towards crashes, multiplayer bugs and controls, later ones towards boss design and balance (Figure 7).

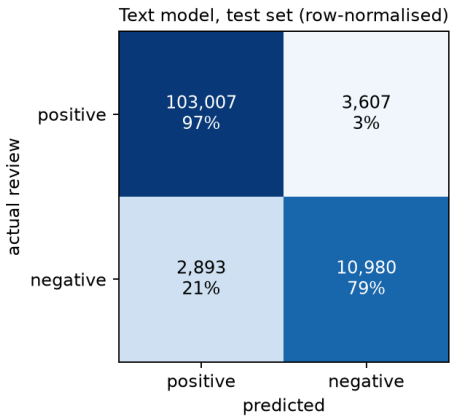


Figure 6. Confusion matrix of the text detector on the 120,487 held-out reviews (counts and row-normalised shares; Python/Matplotlib; python -m src.models.review_text_detail).

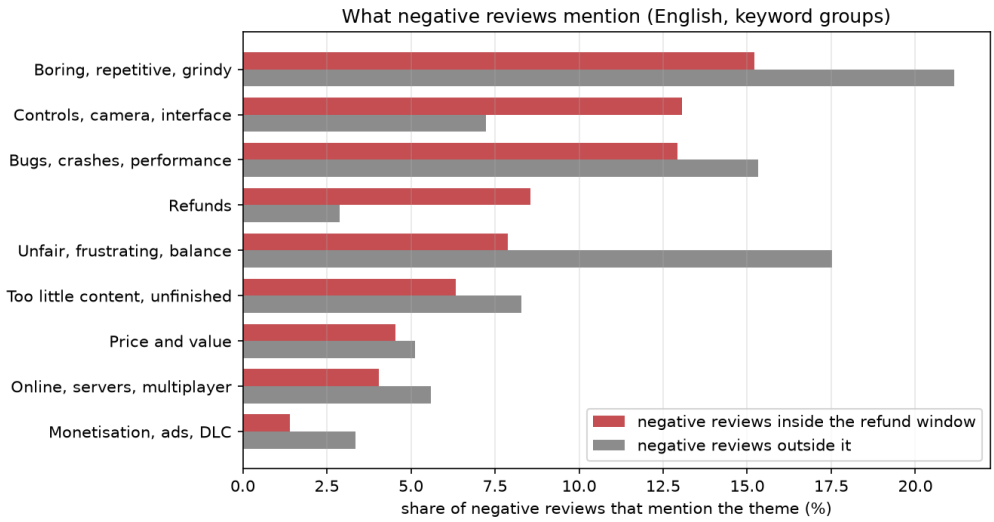


Figure 7. Complaint themes among negative reviews by playtime at review (NMF, eight topics; Python/Matplotlib; notebook 03).

In a blind check of 200 reviews labelled by an AI assistant, the labels matched the model on 35.5% (kappa 0.30): Early Access, repetition and crash topics were recovered well, boss and balance topics were not and are treated as exploratory. An earlier 96% figure was withdrawn because those labels were not independent. The detector finds complaints in existing reviews; it does not forecast them.

5.2 Method 2: Association rule mining

Association rules ask which tag, price and Early Access combinations go with the Strong tier. Each of the 1,191 training games is a transaction of up to 137 items: user tags, a price band and Early Access. Opinion tags such as *Great Soundtrack* were left out because players add them after playing. FP-Growth [13] in mlxtend [16] kept itemsets of up to three items seen in at least 25 games (about 2%); lower supports give rules that rarely hold on new games (Table 7).

Take the strongest rule, *Pixel Graphics + Retro + Shooter leads to Strong*: its support is the 25 training games with all three tags, its confidence the 80% of them that are Strong, and its lift 80% divided by the 35% base rate, 2.26. From 4,715 candidates we kept rules with lift at least 1.2 that passed a false-discovery control (Benjamini-Hochberg, q below 0.10) [14]. A rule holds on test games if, among the test games with all its items, the Strong share is still above the base rate and consistent with training (training confidence inside the 95% test interval); rules matching fewer than 10 test games were not tested.

Table 7. Association-rule configuration comparison (same 1,191 training and 299 test games for every row).

Configuration	Strong rules (FDR)	Mean confidence / lift	Hold on test games
FP-Growth, support 25, up to 3 items (chosen)	95	0.59 / 1.68	68% of 66

Configuration	Strong rules (FDR)	Mean confidence / lift	Hold on test games
Apriori, same thresholds	95	0.59 / 1.68	68% of 66
FP-Growth, support 15 (looser)	107	0.65 / 1.85	66% of 56
FP-Growth, support 40 (stricter)	69	0.57 / 1.60	69% of 59
FP-Growth, pairs only	41	0.56 / 1.59	83% of 30
FP-Growth, keeping after-play tags	131	0.60 / 1.69	69% of 88

95 rules pass for the Strong tier and none for Struggling, so under these thresholds no combination is clearly associated with a struggling launch. About two-thirds of the testable rules (45 of 66) hold on unseen games (Figure 8), and FP-Growth and Apriori return the same rules.

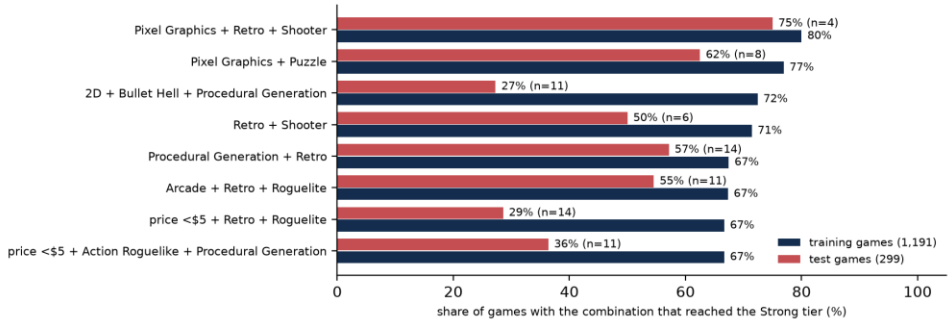


Figure 8. The eight Strong rules with the highest lift: share of Strong games on training and test games (Python/Matplotlib; reports/results/association_rules_strong.csv).

Each rule rests on 25 to 60 training games and few test games, and rules with a price under $5 are often unstable; the rules are hypotheses to test, not recipes, and do not explain why a combination does well.

6. Comparative Analysis and Business Impact

6.1 Comparison of the two Unit 3 methods

Table 8. Comparison of the two selected Unit 3 methods (their metrics are not interchangeable).

Criterion	Method 1: text mining	Method 2: association rules
Question	What do players complain about; can text flag a negative review?	Which launch combinations go with Strong reception?
Input and output	557,202 English reviews; detector scores and eight themes	1,191 training games as itemsets; 95 Strong rules
Validation	Held-out reviews from unseen games; PR-AUC, ROC-AUC; topic diversity and stability; blind check	FDR control; each rule checked once on 299 test games; stability
Strengths / limits	Strong detector (ROC-AUC 0.97) and readable themes / detects rather than forecasts; topic agreement 35.5%; English only	Transparent launch-time rules / small supports; no Struggling rule; associations only

The metrics are not comparable: PR-AUC measures how well the detector separates negative reviews, while lift measures how much a combination raises the chance of a Strong game. Across units the findings agree: complaints cluster in the first hours and at higher prices (Unit 1; crashes

and controls in the text), and store data and tags only partly separate good launches from weak ones (Unit 2 and the rules). As an extra analysis, positive reviews fall by about 2 points after a patch, but matched games without a patch fall almost as much (difference +0.27 points, interval -0.82 to +1.36), so we find no evidence of a patch effect of its own (Appendix B, Figure 13).

6.2 Comparison with existing studies

Table 9. Comparison with existing studies (direct numerical comparison is not valid: different data, targets and protocols).

Criterion	Lin et al. [1]	De Luisa et al. [2]	SteamBR [3]	Current project
Data and setting	About 11M reviews of 6,224 Steam games (2016 snapshot), whole platform	Steam games; popularity as player numbers	2.8M Brazilian-Portuguese Steam reviews	1,861 indie roguelikes, 1.65M reviews, Oct 2026
Methods and validation	Descriptive analysis; no model scored on unseen games	Bayesian hierarchical models; coefficients	Text classifiers for review helpfulness	Models scored once on held-out games; baselines; FDR-controlled rules; topics
Comparable findings	Negative reviews come early (we: 21.2% inside 2 h); design criticised more than bugs (we: crashes early, balance later)	Price, languages and release date matter for popularity; our tier macro-F1 is modest (0.42 to 0.44)	Review text can be modelled; ours detects negativity (PR-AUC 0.84), a different target	Adds refund window, patch timing, text and tags in one niche
Why not numeric	No predictive metric	Different target (players, not tiers)	Helpfulness, other language	Gaps: causal claims, revenue, other genres

No single number compares across these studies. Lin et al.'s finding that negative reviews arrive early matches our refund-window result in direction, and SteamBR supports modelling Steam review text, although for helpfulness in Portuguese. Our contribution is a niche collected from scratch, the refund-window view, held-out testing that reports weak results too, and a usable application.

6.3 KPI-based business impact and scenarios

Measured: about 14% of reviews are written in the first two hours, 21.2% of them negative, against 11.4% for the cheapest games. Assumed: a game with 1,000 reviews; a first-hour fix (blind testing, crash and control fixes) of about 40 person-hours at an illustrative $30 per hour (our assumption, not an industry rate), about $1,200; and an early-negativity drop of 0, 5 or 9.8 points (the last matches the cheapest tier). Sales and refunds are not public, so Table 10 gives the cost per avoided negative review instead of a money benefit.

Table 10. Illustrative scenarios for a first-hour fix (hypothetical effect sizes and cost; sensitivity to the share of reviews written inside the window).

Scenario	Change in early negativity	Avoided negatives per 1,000 reviews (window share 10% / 14% measured / 20%)	Overall negative share (14%)	Cost per avoided negative ($1,200)
Worst	-0.0 points	0.0 / 0.0 / 0.0	10.4% to 10.4%	no benefit
Base	-5.0 points	5.0 / 7.0 / 10.0	10.4% to 9.7%	$171
Best	-9.8 points	9.8 / 13.7 / 19.6	10.4% to 9.0%	$88

The base case avoids about 7 negative reviews per 1,000 at roughly $171 each; the net impact is positive only if an avoided early negative review is worth more than that in kept sales and rating, which our data cannot show. The fix pays off more easily for games expecting many reviews, since the cost is fixed. Patching only to lift sentiment is not supported by our data; patching specific crashes and controls still makes sense.

7. Interactive Application, Figure Index and Reproducibility

7.1 Mandatory dashboard

The dashboard is public at https://indieguard.streamlit.app (the free host sleeps when idle) and runs locally with `streamlit run src/dashboard/app.py`. It is built with Streamlit [17], pandas [18] and Altair [19] and reads small tables exported from the analysis plus one saved XGBoost model. Unit 1 summaries and the Unit 3 rule, topic and patch results are precomputed; filters, charts and the launch-risk scorer are computed live. Its five pages are a market explorer, tag rules and complaints, patch impact, the launch-risk scorer and an about page (Figures 9 to 12).

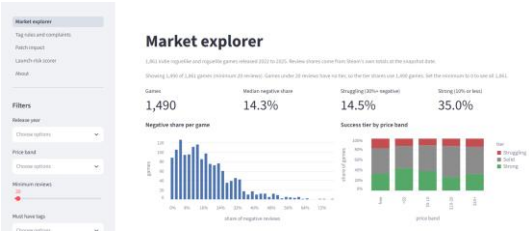


Figure 9. Market explorer, default state (1,490 games).

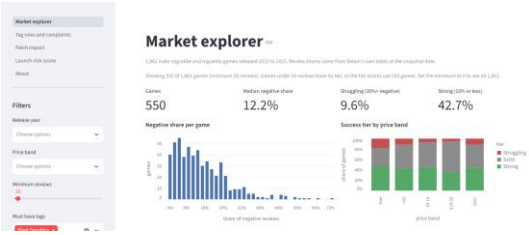


Figure 10. Market explorer after selecting the tag Pixel Graphics (550 games; KPIs and charts update).



Figure 11. Tag rules page filtered to stable rules with lift at least 1.5 (37 of 95 rules).



Figure 12. Scorer explanation for a game predicted Struggling: reasons for and against, matched rules, and single changes that would help.

At Review 2 the panel asked that the scorer explain its output. After scoring, each launch choice (achievements, controller support, a tag and so on) is replaced by the opposite choice and the game is scored again, so the page lists the choices that lower and raise the risk of a Struggling launch, the tag rules the game matches, and the single changes that would help most (Figure 12). Tests: the saved feature pipeline reproduces the stored features for all 1,861 games, and every page was opened in the deployed app and an automated browser run. The scores are class-scaled rather than probabilities (about 0.43 macro-F1), the explanations are associations, and the data are an October 2026 snapshot.

7.2 Figure index and reproduction map

Table 11. Figure index and reproduction map (all figures are Python/Matplotlib or Seaborn and are embedded once).

Figure	Content (section)	Source script, notebook or folder
1	Workflow (1.1)	`src/report/build_capstone_report.py`
2, 14	Refund window; store signals (3, App. B)	notebook 02; `figures/findings`, `figures/eda`
3, 4, 5	Model comparison, confusion, SHAP (4)	`python -m src.models.compare`; `figures/models`
6, 7	Text detector; complaint themes (5.1)	`src.models.review_text_detail`; notebook 03
8	Rules on train and test games (5.2)	`reports/results/association_rules_strong.csv`
9 to 12	Dashboard states (7.1)	`figures/dashboard`
13	Patch impact (6.1, App. B)	`notebooks/review2/patch_impact.ipynb`

7.3 Code and environment

The code is public at https://github.com/Shan713/IndieGuard; the README describes the folder layout (`src`, `notebooks`, `data`, `figures`, `reports`, `docs`, `models`). The project runs on Python

3.12 with versions pinned in `requirements.txt` (`pip install -r requirements.txt`), and Appendix C lists the commands that rebuild every result. No secrets are stored; the raw scrape and the hashing salt are left out, and the collection scripts can regenerate the data from Steam's public endpoints.

8. End-to-End Validation, Traceability and Contributions

8.1 Completion and validity

The declared scope is fully implemented (Table 12). The main threats to validity are the small number of training games for tuning (two models overfitted their folds) and a topic check with one AI annotator (internal); one niche, one snapshot, about 40% English reviews and no revenue data (external); and reviewer history collected at scrape time and a tier defined from the negative share (construct). Game-level scores move by about 0.01 between runs.

Table 12. Objective-to-evidence traceability matrix.

Objective	Method (section)	Key result	Evidence	Rec.
O1	EDA (3)	21.2% vs 8.8% negative; 29% of negatives	Fig. 2	R1, R2
O2	Classification (4, 5.1)	PR-AUC 0.838, ROC-AUC 0.971 (guessing 0.115)	Table 6	R1
O3	Game-tier models (4)	Macro-F1 0.42 to 0.44 vs 0.22	Table 5	R4
O4	Association rules (5.2)	95 Strong rules; 68% of 66 testable hold	Table 7, Fig. 8	R3
O5	Topic modelling (5.1)	Diversity 90%; blind agreement 35.5%	Fig. 7	R1, R5
O6	Matched controls (6.1)	+0.27 points [-0.82, +1.36]	Fig. 13	R5
O7	Dashboard (7)	Five working pages, filters and scorer	Figs. 9 to 12	R1 to R5

8.2 Project tracking and individual work

Work was tracked on a GitHub Projects board (https://github.com/users/Shan713/projects/1) with the stages Backlog, To Do, In Progress, Review/Testing and Completed; each card has an owner, due date and evidence (cards per member in Appendix D). Individual marks may vary with participation and the explanation of assigned work.

Table 13. Individual contributions, dates and evidence (dates from the pull-request history).

Member / register no.	Assigned work	Period (2026)	Main evidence
Thotakura Srihitha / CB.SC.U4CSE23450	Exploratory analysis (29 figures, association tests); dashboard wireframe and downloads; grid-search tuning	2 to 17 Oct	PRs #1, #101, #102; notebook 02
Gayas Sheik / CB.SC.U4CSE23118	Feature engineering and tag PCA; patch impact with matched controls; launch-risk scorer; reproducibility	2 to 14 Oct	PRs #2, #49, #50, #69, #71, #80; `src/features`
Shantharam P / CB.SC.U4CSE23146	Data collection and cleaning; split and targets; grid-search tuning; dashboard and deployment	1 to 17 Oct	PRs #57, #95, #96, #102; `src/collect`, `src/clean`

Member / register no.	Assigned work	Period (2026)	Main evidence
Pradeep Sasidharan / CB.SC.U4CSE23232	Model development (Logistic Regression, Random Forest, XGBoost; SHAP); model comparison; association rules	3 to 11 Oct	PRs #55, #70; `src/models`
Valikala Tejaswini / CB.SC.U4CSE23752	Text mining and topic modelling; combined business insights; blind topic check	10 to 16 Oct	PRs #66, #81, #99; notebook 03

8.3 AI use and originality declarations

We used three generative-AI tools: ChatGPT for ideation, Antigravity for minor code completions and fixes, and Claude for coding and debugging help, drafting documentation, this report and the deck, and labelling the blind topic-check sample. Appendix D (Table 20) gives the complete disclosure: tasks, content and code affected, and how the team verified and corrected each output. No tool collected data, altered records or decided conclusions; every figure is produced by code from the project data. Apart from cited sources, the ideas, analysis and conclusions are the team's own, and AI-drafted text or code was reviewed, edited and verified by the team before use.

9. Conclusion, Recommendations and Roadmap

The traceability matrix (Table 12) shows how each objective was met. Three results were weaker than we hoped: game-tier prediction from store data is modest, no rule was found for struggling games, and patches showed no effect. They show where store data and patching alone are not enough. Future work, not done here, covers discounts, causal designs, revenue, a language comparison for the review models and a human-labelled topic check. Table 14 gives the recommendations, all hypotheses to test.

Table 14. Prioritised recommendations and implementation roadmap (in priority order).

Recommendation (hypothesis)	KPI	Owner	Timing, resources and success check
R1. Treat the first hour as a standalone product: blind-test it, check controls, enable crash reporting	Early negative share (21.2%)	QA lead	Now; 40 person-hours, 10 to 20 testers. Check: below 21%
R2. Match price to what the first hour delivers (early negativity 11% under $5, 32% at $20+)	Early negativity by price band	Publisher	Now; small price test. Check: band's early negativity
R3. Test a focused 2D, pixel, retro or procedural scope before multiplayer or 3D	Strong share (2/3 vs 35%)	Design lead	1 to 6 months. Check: 2/3 of rules still hold
R4. Test whether achievements and controller support help (12.5% vs 19.6% is observational)	Struggling share	Marketing	1 to 6 months. Check: Struggling share below 14.5%
R5. Patch specific problems (crashes, controls, balance), not to lift sentiment	Change vs controls	Engineering	Ongoing; matched-control check per patch
Analytics: refresh data, hand-label 200 reviews, calibrate the scorer	Agreement; macro-F1	Analytics	6 to 12 months. Targets: agreement above 35.5%; macro-F1 0.43+

References

[1] Dayi Lin, Cor-Paul Bezemer, Ying Zou, and Ahmed E. Hassan. 2019. An empirical study of game reviews on the Steam platform. *Empirical Software Engineering* 24, 1 (2019), 170-207. https://doi.org/10.1007/s10664-018-9627-4

[2] Andraž De Luisa, Jan Hartman, David Nabergoj, Samo Pahor, Marko Rus, Bozhidar Stevanoski, Jure Demšar, and Erik Štrumbelj. 2021. Predicting the popularity of games on Steam. *Elektrotehniški vestnik* 88, 4 (2021), 151-162. arXiv:2110.02896.

[3] Gabriel A. Z. Jorge and Thiago A. S. Pardo. 2023. SteamBR: a dataset for game reviews and evaluation of a state-of-the-art method for helpfulness prediction. In *Proceedings of the Brazilian Workshop on Social Network Analysis and Mining (BraSNAM)*.

[4] Valve Corporation. Steam Refunds. Steam Support. https://store.steampowered.com/steam_refunds/ (accessed October 2026); and User Reviews, Steamworks documentation. https://partner.steamgames.com/doc/store/getreviews (accessed October 2026).

[5] SteamSpy. SteamSpy API: owner estimates and game statistics, with request-rate limits. https://steamspy.com/api.php (accessed October 2026).

[6] Government of India. 2023. The Digital Personal Data Protection Act, 2023 (No. 22 of 2023).

[7] Fabian Pedregosa et al. 2011. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825-2830.

[8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*.

[9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: a scalable tree boosting system. In *Proceedings of KDD '16*. ACM, 785-794. https://doi.org/10.1145/2939672.2939785

[10] Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management* 24, 5 (1988), 513-523.

[11] Daniel D. Lee and H. Sebastian Seung. 1999. Learning the parts of objects by non-negative matrix factorization. *Nature* 401 (1999), 788-791.

[12] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet allocation. *Journal of Machine Learning Research* 3 (2003), 993-1022.

[13] Jiawei Han, Jian Pei, and Yiwen Yin. 2000. Mining frequent patterns without candidate generation. In *Proceedings of SIGMOD '00*. ACM, 1-12. https://doi.org/10.1145/335191.335372

[14] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B* 57, 1 (1995), 289-300.

[15] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*.

[16] Sebastian Raschka. 2018. MLxtend: providing machine learning and data science utilities and extensions to Python's scientific computing stack. *Journal of Open Source Software* 3, 24 (2018), 638.

[17] Streamlit Inc. Streamlit: the fastest way to build data apps. https://streamlit.io (version 1.66, accessed October 2026).

[18] Wes McKinney. 2010. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, 56-61. https://doi.org/10.25080/Majora-92bf1922-00a

[19] Jacob VanderPlas, Brian E. Granger, Jeffrey Heer, Dominik Moritz, Kanit Wongsuphasawat, Arvind Satyanarayan, Eitan Lees, Ilia Timofeev, Ben Welsh, and Scott Sievert. 2018. Altair: interactive statistical visualizations for Python. *Journal of Open Source Software* 3, 32 (2018), 1057. https://doi.org/10.21105/joss.01057

[20] John D. Hunter. 2007. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering* 9, 3 (2007), 90-95. https://doi.org/10.1109/MCSE.2007.55

[21] Michael L. Waskom. 2021. seaborn: statistical data visualization. *Journal of Open Source Software* 6, 60 (2021), 3021. https://doi.org/10.21105/joss.03021

[22] Valve Corporation. Steam Web API Terms of Use. https://steamcommunity.com/dev/apiterms (accessed October 2026).

In-text citation numbers follow ACM numeric style.

Appendix A. Extended data dictionary and validation records

Table 15 lists the main fields of the three cleaned tables. The full dictionary (all 34 review columns, 58 game columns and the event table) is in `docs/data_documentation.md`, section 5, and the feature catalogue is `docs/feature_engineering/feature_catalog.csv`.

Table 15. Concise data dictionary of the cleaned tables.

Table	Field	Type	Meaning
Review	`recommendationid`	int	Steam's review ID (primary key)
Review	`appid`	int	Game ID; joins to the game table
Review	`voted_up`	bool	Recommended (True) or not; the review target is not voted_up
Review	`review`, `language`	text, category	Review text as written; 31 languages (39.5% English)
Review	`author_playtime_at_review`	float (minutes)	Playtime when the review was written (12 missing)
Review	`created`	datetime (UTC)	When the review was written
Review	`steam_purchase`, `received_for_free`	bool	Bought on Steam; marked as received for free (2.1%)
Review	`written_during_early_access`	bool	Written while the game was in Early Access (22.7%)
Review	`author_steamid_hash`	text	Salted SHA-256 of the reviewer ID (pseudonymised)
Game	`appid`, `name`	int, text	Steam app ID and name
Game	`price_usd`, `is_free`	float, bool	Regular price in USD (0 if free); free to play (308 games)
Game	`launch_date`, `release_date`	datetime	Original launch including Early Access; store (1.0) release
Game	`tags`, `genres`, `categories`	text lists	Top 20 user tags; Steam genres; store features
Game	`n_dlc`, `n_achievements`, `n_supported_languages`	int	Counts at collection time (missing achievements = none)
Game	`platform_mac`, `platform_linux`, `controller_support`	bool, text	Platforms; full or partial controller support
Game	`steam_total_reviews`, `steam_total_negative`	int	Steam's all-language totals; used for the tier, never as features
Game	`spy_owners_low`, `spy_owners_high`	int	SteamSpy owner range (outcome; not a feature)
Event	`appid`, `date`, `event_type`	int, datetime, text	Steam news item: patch or sale announcement

Validation records are in `docs/cleaning_log.md`, `docs/validation_summary.md` and `data/processed/cleaning_log.json`. Key flags added by cleaning:

Table 16. Key cleaning flags.

Field	Type	Meaning and count
`playtime_outlier`	bool	Playtime at review above the 99.5% quantile (363 h); 8,242 reviews
`is_prerelease`	bool	Written before the game's launch; 30,018 reviews
`low_content`	bool	Under 3 letters or 2 words; 208,724 reviews
`is_ascii_art`, `is_copypasta`	bool	ASCII art (4,951); same 50+ character text 10+ times (2,285)
`text_mining_ok`	bool	English and none of the three flags: the text-mining subset
`received_for_free`	bool	Reviewer received the game for free; 34,394 reviews
`launch_date` vs `release_date`	datetime	Original launch vs 1.0 date; differ for 318 games
`reviews_capped`	bool	Newest ~15,000 reviews only (5 games)

Appendix B. Tuning, additional figures and supplementary analysis

Table 17. Grid-search results: default (grid point 0) against tuned settings on the shared folds.

Search	Metric	Grid points	Default CV	Tuned CV
Review level, Logistic Regression	PR-AUC	11	0.208	0.208
Review level, Random Forest	PR-AUC	13	0.278	0.281
Review level, LightGBM	PR-AUC	25	0.281	0.287
Review level, XGBoost	PR-AUC	37	0.285	0.292
Game level, Logistic Regression	macro-F1	12	0.418	0.422
Game level, Random Forest	macro-F1	60	0.354	0.434
Game level, LightGBM	macro-F1	55	0.417	0.445
Game level, XGBoost	macro-F1	97	0.409	0.444
Review text, Logistic Regression	PR-AUC	30	0.792	0.794
Review text, LightGBM metadata	PR-AUC	25	0.332	0.334
Review text, LightGBM metadata + text	PR-AUC	25	0.574	0.592

Table 18. Subgroup error check on the held-out test data (game scorer: macro-F1; text detector: PR-AUC; python -m src.models.subgroup_check).

Model	Subgroup	Test n	Macro-F1 / PR-AUC	Struggling recall (n) / precision, recall
Game scorer	all: all test games	299	0.43	0.29 (51)
Game scorer	price: under $5	129	0.42	0.36 (25)
Game scorer	price: $5 to $14.99	133	0.41	0.19 (16)
Game scorer	price: $15 or more	37	0.45	0.30 (10)
Game scorer	reviews: 20 to 99	171	0.40	0.38 (37)
Game scorer	reviews: 100 to 499	65	0.30	0.09 (11)
Game scorer	reviews: 500 or more	63	0.45	0.00 (3)
Game scorer	year: 2022-2023	129	0.36	0.29 (24)
Game scorer	year: 2024-2025	170	0.47	0.30 (27)
Game scorer	early access: Full release	239	0.44	0.31 (45)
Game scorer	early access: Early Access	60	0.33	0.17 (6)
Text detector	window: inside 2 h	15,316	0.91	0.79 / 0.89
Text detector	window: after 2 h	105,171	0.80	0.73 / 0.75
Text detector	price: under $5	4,884	0.86	0.78 / 0.79
Text detector	price: $15 or more	71,864	0.83	0.77 / 0.77
Text detector	price: $5 to $14.99	40,176	0.86	0.74 / 0.84
Text detector	price: free	3,563	0.83	0.60 / 0.87
Text detector	length: 10 to 49 words	49,846	0.81	0.75 / 0.73
Text detector	length: under 10 words	33,022	0.73	0.72 / 0.64
Text detector	length: 50+ words	37,619	0.87	0.76 / 0.86
Text detector	free copy: bought	118,111	0.84	0.75 / 0.79
Text detector	free copy: free copy	2,376	0.83	0.75 / 0.77

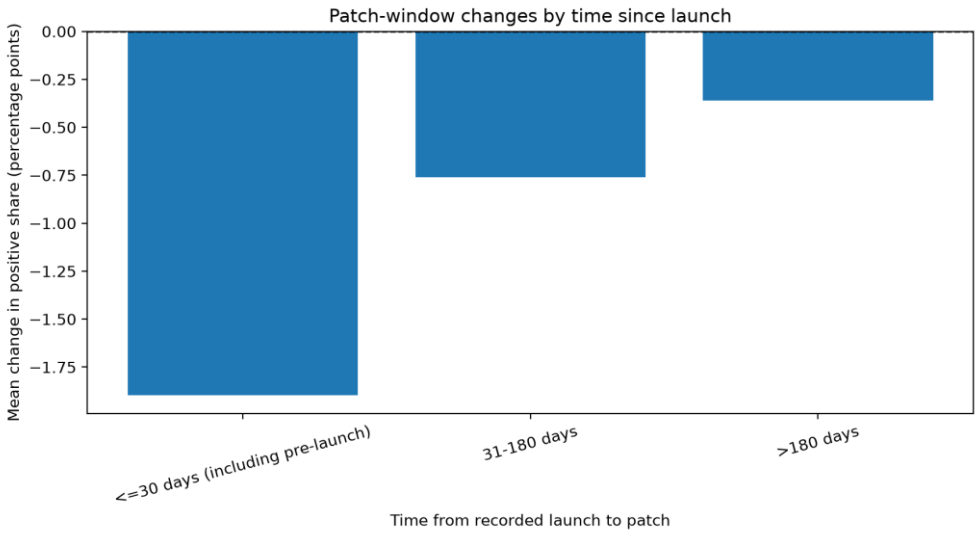


Figure 13. Patch impact by launch timing (change in positive-review share, points; Python/Matplotlib; notebooks/review2/patch_impact.ipynb).

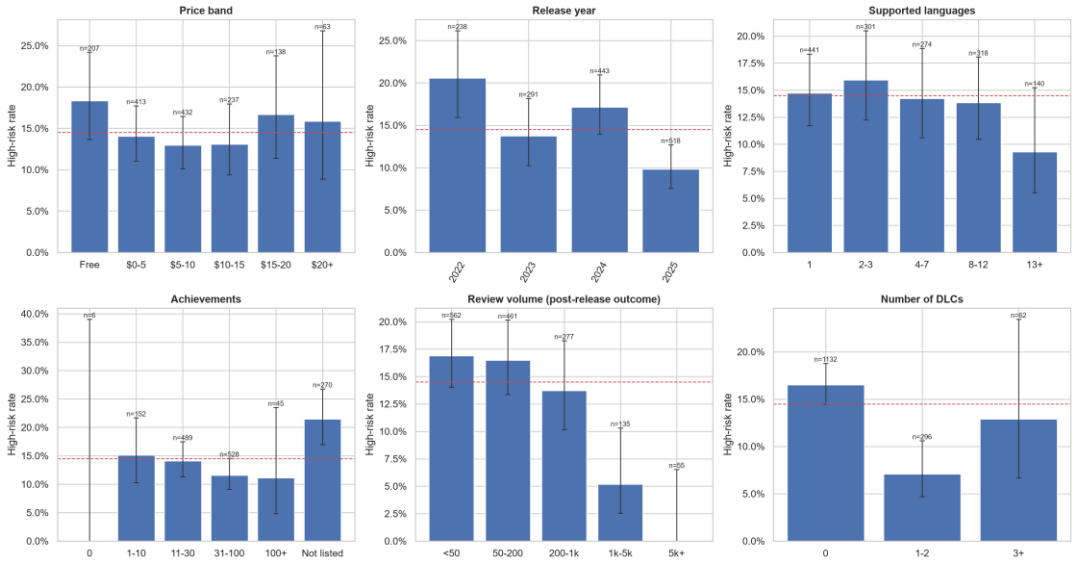


Figure 14. Share of Struggling games by price band, release year, languages, DLC, achievements and review volume with 95% intervals (Python/Seaborn; notebooks/02_EDA_Visualizations.ipynb).

Appendix C. Implementation details and dashboard user guide

Requirements: Python 3.12 and the pinned packages in `requirements.txt`. From the repository root:

- **Install:** `python -m venv .venv ; pip install -r requirements.txt`
- **Features and checks:** `python -m src.features`
- **Models:** `python -m src.models.review_models` (also `game_models`, `review_xgboost`, `game_xgboost`)
- **Model tables:** `python -m src.models.compare`
- **Tuning:** `python -m src.models.grid_search --task game --model xgboost`
- **Subgroup check:** `python -m src.models.subgroup_check`
- **Report:** `python -m src.report.build_capstone_report`

- **Dashboard:** `streamlit run src/dashboard/app.py`

User guide: *Market explorer*: filter by release year, price band, minimum reviews and tags; KPI cards and charts update. *Tag rules and complaints*: set stability, minimum lift and a tag or price to filter rules; open the complaint-themes and refund-window tabs. *Patch impact*: read the raw and matched-control results. *Launch-risk scorer*: describe a planned game (price, launch date, platforms, tags, categories) or load an existing game and press *Score this game*; read the predicted tier and scores (not probabilities).

Appendix D. Tracking-board evidence, contribution records and AI declaration

Project board: https://github.com/users/Shan713/projects/1 (68 cards, all Completed; history, owners, due dates and evidence). Pull-request history: https://github.com/Shan713/IndieGuard/pulls. The originality and AI-use declaration is signed by all members on the declaration page.

Table 19. Project-board cards by member.

Member	Board cards (owner)
Thotakura Srihitha	#20, #21, #37, #40, #41, #86 to #89, #91, #93
Gayas Sheik	#4, #18, #19, #38, #39, #42
Shantharam P	#5 to #17, #23 to #27, #43, #83 to #94
Pradeep Sasidharan	#34, #35, #36
Valikala Tejaswini	#31, #32, #33, #44

Table 20. Generative-AI disclosure (complete).

Tool	Tasks supported	Content / code affected	Human verification and corrections
ChatGPT (OpenAI)	Ideation: brainstorming project topics, research questions and feature ideas in the planning stage.	Early ideas and the shortlist of topics and questions. No data, code or reported results.	The team discussed the ideas, checked feasibility against the available public data, and wrote and agreed the problem statement and KPIs. Nothing from the tool is reported as a result.
Antigravity (AI coding assistant)	Minor coding assistance: small code completions and fixes in project scripts.	Small, scattered completions and fixes; no complete module or analysis.	Each change was read, run and tested by the team member who made it; outputs were checked against expected values and the automated checks (Section 8.1).
Claude (Anthropic)	Coding and debugging help under the team's direction (data collection, modelling, dashboard and report scripts); drafting documentation, this report and the presentation deck; labelling the blind topic-check sample.	Parts of the project scripts, documentation, slides and this report.	The team ran, checked and corrected all outputs: automated checks, results regenerated from scripts, pull-request review, and slides and figures checked against the data. Errors found and fixed included a text-processing bug and an overstated topic-agreement figure, which was withdrawn.